

TRABAJO FINAL INTEGRADOR

Segunda Entrega - Diseño y Módulos

FixTrack

Plataforma SaaS multi-organización para gestión y trazabilidad integral de servicios técnicos

Dato	Información
Carrera	Tecnicatura Universitaria en Programación a Distancia
Integrantes	Matías Ariel Deluca · Gastón Armando Giorgio · Andrés Emanuel Meshler
Tutora	María Candela Grosso
Fecha de entrega	27/09/2026
Repositorio GitHub	https://github.com/ameshler/TFI-SaaS-Servicios-Tecnicos.git
Nombre comercial	FixTrack

Alcance de esta instancia

Esta entrega presenta el diseño inicial de la base de datos, la arquitectura y los módulos funcionales del MVP. No implica que todas las funcionalidades se encuentren implementadas; la implementación continuará de acuerdo con el plan de trabajo del proyecto.

1. Introducción

El presente documento corresponde a la Segunda Entrega del Trabajo Final Integrador de FixTrack y desarrolla el diseño inicial de la base de datos, la arquitectura del proyecto, el listado de módulos funcionales y la organización prevista del repositorio de la plataforma SaaS para gestión de servicios técnicos.

La propuesta mantiene el enfoque aprobado en la primera entrega: una plataforma web multi-organización en la que distintos servicios técnicos utilizan una misma aplicación, manteniendo sus datos lógicamente aislados. El MVP contempla autenticación, organizaciones, usuarios y roles, clientes, equipos, órdenes de reparación, asignación de técnicos, diagnóstico, presupuesto, estados, historial, entrega y dashboard básico.

Objetivo de esta entrega

Validar un modelo de datos documental coherente con el flujo de negocio, la arquitectura elegida y los módulos funcionales del MVP, dejando una base consistente para la etapa de implementación.

2. Arquitectura del proyecto

2.1. Enfoque arquitectónico

Se utilizará una arquitectura cliente-servidor. El frontend será una aplicación web desarrollada con React + TypeScript y consumirá mediante HTTPS/JSON una API REST desarrollada con Java + Spring Boot.

El backend se organizará mediante una arquitectura por capas, separando responsabilidades de presentación, lógica de negocio y persistencia. La estructura conceptual será Controller → Service → Repository → MongoDB Atlas. Spring Security y JWT actuarán como componentes transversales para autenticación y autorización.

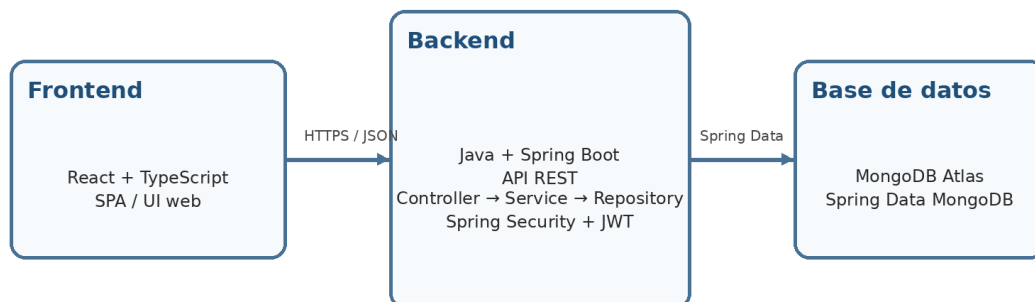


Figura 1. Arquitectura preliminar de la solución.

2.2. Tecnologías definitivas y justificación

Capa	Tecnología	Justificación
Frontend	React + TypeScript	Interfaz web modular y mantenible. TypeScript aporta tipado estático y facilita la detección temprana de errores.
Backend	Java + Spring Boot	Framework maduro para APIs REST con lógica de negocio, validación, seguridad e integración con el ecosistema Spring.
Persistencia	Spring Data MongoDB	Permite trabajar con documentos y repositorios de MongoDB desde Spring, sin utilizar JPA/Hibernate sobre una base NoSQL.
Base de datos	MongoDB Atlas	Base documental administrada en la nube, adecuada para el agregado de órdenes y disponible como servicio online.
Seguridad	Spring Security + JWT	Autenticación y autorización stateless, permisos por rol y protección de endpoints.
Despliegue previsto	Vercel + Render + MongoDB Atlas	Separación simple de frontend, backend y datos para la etapa de implementación y despliegue.
Versionado	GitHub	Repositorio único para documentación y futuras implementaciones del equipo.

2.3. ¿Por qué MongoDB y no una base de datos relacional?

El dominio posee relaciones claras y podría implementarse con una base relacional. Sin embargo, se seleccionó MongoDB porque la orden de reparación funciona como agregado principal del sistema y concentra información estrechamente vinculada a su ciclo de vida, como diagnóstico, presupuesto, observaciones, historial de estados y datos de entrega.

Estos elementos suelen consultarse junto con la orden y pueden representarse como subdocumentos cuando resulte conveniente. En cambio, las entidades con ciclo de vida propio —organizaciones, usuarios, clientes y equipos— se mantendrán como documentos independientes y se vincularán mediante identificadores. La elección busca aprovechar el modelo documental en el agregado central sin eliminar la separación lógica entre entidades principales.

3. Arquitectura de datos

3.1. Tipo de base de datos

Se utilizará MongoDB Atlas como base de datos NoSQL orientada a documentos. El modelo combinará referencias entre documentos independientes con subdocumentos embebidos dentro de las órdenes de reparación.

3.2. Colecciones principales

Colección	Responsabilidad
organizations	Representa cada servicio técnico registrado en la plataforma.
users	Usuarios pertenecientes a una organización y sus roles.
clients	Clientes que ingresan uno o más equipos al servicio técnico.
equipment	Equipos asociados a clientes y a una organización.
repair_orders	Órdenes de reparación y ciclo completo de atención.
audit_logs	Registro técnico previsto para acciones relevantes de trazabilidad y auditoría; no constituye un módulo funcional adicional.

3.3. Estrategia de documentos embebidos y referencias

La estrategia combina documentos independientes con información embebida. Se utilizarán referencias para entidades reutilizables o administradas de forma independiente, mientras que la información propia del ciclo de vida de una reparación permanecerá dentro de `repair_orders`.

Tipo	Elementos	Criterio
Referenciados	Organization, User, Client, Equipment	Entidades con identidad y ciclo de vida propios, consultadas y administradas independientemente.
Embebidos	Diagnosis, Quote, StatusHistory, Delivery	Información dependiente de la orden y normalmente consultada junto con ella.

4. Modelo documental

4.1. Organizations

Representa un servicio técnico dentro de la plataforma.

```
{
  "_id": "ObjectId",
  "name": "Servicio Técnico Centro",
  "createdAt": "Date",
  "active": true
}
```

Campo	Tipo	Obligatorio	Descripción
<code>_id</code>	ObjectId	Sí	Identificador de la organización.
<code>name</code>	String	Sí	Nombre del servicio técnico.
<code>createdAt</code>	Date	Sí	Fecha de creación.
<code>active</code>	Boolean	Sí	Indica si la organización se encuentra activa.

4.2. Users

Los usuarios trabajan dentro de una única organización en el MVP y poseen un rol.

```
{
  "_id": "ObjectId",
  "organizationId": "ObjectId",
  "name": "Juan Pérez",
  "email": "juan@serviciotecnico.com",
  "passwordHash": "...",
  "role": "TECHNICIAN",
  "active": true,
  "createdAt": "Date"
}
```

Campo	Tipo	Obligatorio	Descripción
_id	ObjectId	Sí	Identificador del usuario.
organizationId	ObjectId	Sí	Organización a la que pertenece.
name	String	Sí	Nombre del usuario.
email	String	Sí	Correo utilizado para autenticación; será único globalmente en el MVP.
passwordHash	String	Sí	Contraseña almacenada mediante hash.
role	Enum/String	Sí	ADMIN, RECEPTIONIST o TECHNICIAN.
active	Boolean	Sí	Estado lógico del usuario.
createdAt	Date	Sí	Fecha de alta.

Decisión de autenticación

Para simplificar el inicio de sesión del MVP, cada email será único en toda la plataforma. El login podrá resolverse con email + contraseña sin solicitar un identificador adicional de organización. El usuario seguirá asociado a una única organizationId.

4.3. Clients

Representa al cliente que entrega uno o más equipos al servicio técnico.

```
{
  "_id": "ObjectId",
  "organizationId": "ObjectId",
  "name": "Carlos Gómez",
  "phone": "342-5555555",
  "email": "carlos@email.com",
  "createdAt": "Date",
  "active": true
}
```

Campo	Tipo	Obligatorio	Descripción
_id	ObjectId	Sí	Identificador del cliente.
organizationId	ObjectId	Sí	Organización a la que pertenece el cliente.
name	String	Sí	Nombre o razón social del cliente.
phone	String	No	Teléfono de contacto.
email	String	No	Correo electrónico de contacto.
createdAt	Date	Sí	Fecha de alta.
active	Boolean	Sí	Estado lógico del cliente.

4.4. Equipment

Representa un equipo perteneciente a un cliente.

```
{
  "_id": "ObjectId",
  "organizationId": "ObjectId",
  "clientId": "ObjectId",
  "type": "Notebook",
  "brand": "Lenovo",
  "model": "ThinkPad E14",
  "serialNumber": "ABC123456",
  "createdAt": "Date",
  "active": true
}
```

Campo	Tipo	Obligatorio	Descripción
_id	ObjectId	Sí	Identificador del equipo.
organizationId	ObjectId	Sí	Organización a la que pertenece el equipo.
clientId	ObjectId	Sí	Cliente propietario del equipo.
type	String	Sí	Tipo de equipo.
brand	String	Sí	Marca.
model	String	Sí	Modelo.
serialNumber	String	No	Número de serie, cuando exista.
createdAt	Date	Sí	Fecha de alta.
active	Boolean	Sí	Estado lógico del equipo.

4.5. Repair Orders

repair_orders constituye la colección central del sistema. Contiene referencias a organización, cliente, equipo y técnico, junto con la información propia del proceso de reparación.

```
{
  "_id": "ObjectId",
  "organizationId": "ObjectId",
  "orderNumber": "ORD-000125",
  "clientId": "ObjectId",
  "equipmentId": "ObjectId",
  "assignedTechnicianId": "ObjectId",
  "status": "IN_DIAGNOSIS",
  "problemDescription": "El equipo no enciende",
  "diagnosis": { ... },
  "quote": { ... },
  "statusHistory": [ ... ],
  "delivery": { ... },
  "createdAt": "Date",
  "updatedAt": "Date"
}
```

Campo	Tipo	Obligatorio	Descripción
_id	ObjectId	Sí	Identificador interno de la orden.
organizationId	ObjectId	Sí	Organización a la que pertenece la orden.
orderNumber	String	Sí	Número visible de la orden, único dentro de cada organización.
clientId	ObjectId	Sí	Cliente asociado.
equipmentId	ObjectId	Sí	Equipo asociado.
assignedTechnicianId	ObjectId	No	Técnico responsable actual.
status	Enum	Sí	Estado operativo actual de la orden.
problemDescription	String	Sí	Problema informado al ingresar el equipo.
diagnosis	Object	No	Diagnóstico técnico y observaciones.
quote	Object	No	Presupuesto y su estado.
statusHistory	Array	Sí	Historial cronológico de cambios de estado.
delivery	Object	No	Datos de cierre y entrega.
createdAt	Date	Sí	Fecha de creación.
updatedAt	Date	Sí	Última modificación.

4.5.1. Subdocumentos de Repair Orders

Subdocumento	Campos principales
Diagnosis	problemDetected, proposedSolution, observations, technicianId, createdAt
Quote	description, amount, status, createdAt
StatusHistory	status, changedBy, changedAt, observation
Delivery	deliveredAt, deliveredBy, observation

4.6. Audit Logs

La colección de auditoría se prevé como componente técnico transversal para registrar acciones relevantes relacionadas con seguridad, cambios de estado y operaciones sensibles. No constituye un módulo funcional adicional del MVP y podrá implementarse de manera incremental durante el desarrollo.

```
{
  "_id": "ObjectId",
  "organizationId": "ObjectId",
  "userId": "ObjectId",
  "action": "ORDER_STATUS_CHANGED",
  "entity": "REPAIR_ORDER",
  "entityId": "ObjectId",
  "createdAt": "Date"
}
```

Campo	Tipo	Obligatorio	Descripción
_id	ObjectId	Sí	Identificador del registro.
organizationId	ObjectId	Sí	Organización a la que corresponde la acción.
userId	ObjectId	Sí	Usuario que ejecutó la acción.
action	String/Enum	Sí	Tipo de acción registrada.
entity	String/Enum	Sí	Entidad afectada.
entityId	ObjectId	No	Identificador de la entidad afectada, cuando corresponda.
createdAt	Date	Sí	Fecha y hora de la acción.

5. Relaciones conceptuales

Aunque MongoDB no utiliza relaciones como una base relacional tradicional, los documentos se vincularán mediante identificadores. Las relaciones conceptuales principales son:

- Organization 1:N Users
- Organization 1:N Clients
- Organization 1:N Equipment
- Organization 1:N Repair Orders
- Organization 1:N Audit Logs
- Client 1:N Equipment
- Client 1:N Repair Orders
- Equipment 1:N Repair Orders
- User (Técnico) 1:N Repair Orders

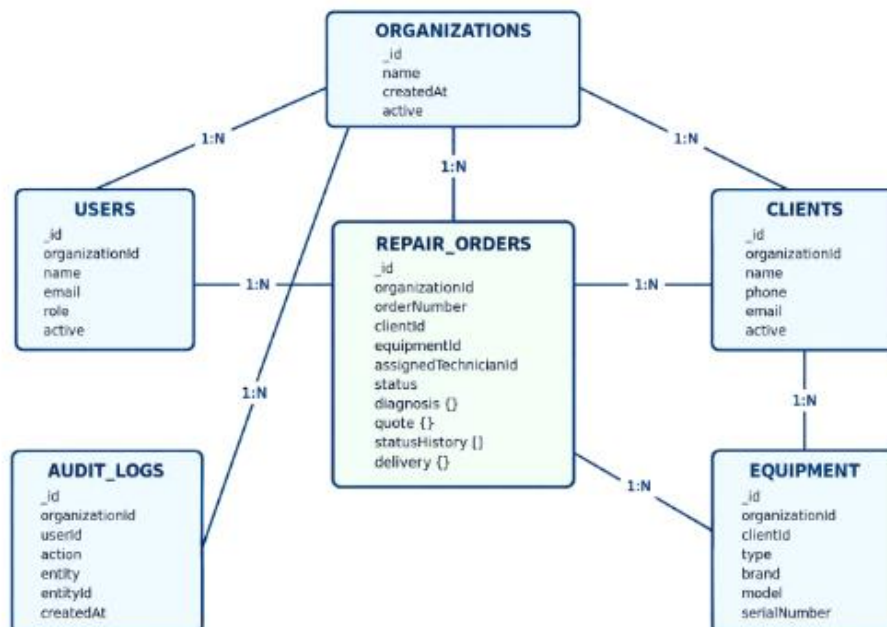


Figura 2. Esquema conceptual de colecciones, referencias y agregado principal.

6. Ciclo de vida de una orden

El flujo mantiene el definido en la primera entrega. Para la futura implementación se utilizarán nombres técnicos de estado coherentes con el código, mientras que la interfaz podrá mostrarlos en español.

Estado técnico	Descripción
RECEIVED	Registro inicial del equipo y creación de la orden.
IN_DIAGNOSIS	Análisis técnico y registro del problema detectado.
AWAITING_BUDGET_APPROVAL	Presupuesto cargado y pendiente de decisión del cliente.
IN_REPAIR	Presupuesto aprobado y trabajo técnico en ejecución.
READY_FOR_DELIVERY	La reparación fue finalizada y el equipo queda disponible para retiro.
DELIVERED	Cierre de la orden y registro de entrega.
REJECTED	Salida alternativa cuando el presupuesto no es aprobado; la orden no continúa hacia reparación.

6.1. Estado de la orden y estado del presupuesto

El estado operativo de la orden y el estado del presupuesto son conceptos diferentes, pero deben mantener consistencia entre sí.

Elemento	Valores	Responsabilidad
OrderStatus	RECEIVED, IN_DIAGNOSIS, AWAITING_BUDGET_APPROVAL, IN_REPAIR, READY_FOR_DELIVERY, DELIVERED, REJECTED	Representa el estado operativo actual de la orden.
QuoteStatus	PENDING, APPROVED, REJECTED	Representa exclusivamente la situación del presupuesto.

Regla de consistencia

Una orden solo podrá pasar de AWAITING_BUDGET_APPROVAL a IN_REPAIR cuando quote.status sea APPROVED. Si quote.status es REJECTED, la orden pasará a REJECTED. Cada cambio relevante deberá generar una entrada en statusHistory con usuario responsable, fecha/hora y observación cuando corresponda.

7. Aislamiento multi-organización

El aislamiento lógico constituye un requisito transversal del sistema. Todos los documentos pertenecientes a un servicio técnico incluirán organizationId. La colección organizations constituye la raíz de cada tenant y por ello no requiere un organizationId propio.

Regla	Aplicación
Identificación	Todo documento de negocio perteneciente a una organización contiene organizationId.
Autenticación	JWT identifica al usuario autenticado.
Contexto	El backend obtiene la organización desde el usuario autenticado; no confía en un organizationId enviado libremente por el frontend.
Consultas	Los repositorios y servicios filtran por organizationId.
Acceso individual	Las búsquedas por id deben validar también la organización, por ejemplo findByIdAndOrganizationId(id, organizationId).
Autorización	Los roles restringen operaciones sensibles, pero no reemplazan el aislamiento entre tenants.
Integridad	Cliente, equipo, técnico y orden asociados deben pertenecer a la misma organización.
Pruebas	Se contemplarán casos negativos de acceso cruzado entre organizaciones durante la implementación.

8. Índices y validaciones preliminares

Los índices son preliminares y podrán ajustarse según las consultas reales de la aplicación. Se priorizan filtros por tenant y los accesos más frecuentes del MVP.

Colección	Índice	Objetivo
users	{ email: 1 } UNIQUE	Garantizar que el email utilizado para login sea único en toda la plataforma.
users	{ organizationId: 1, role: 1 }	Listar usuarios de una organización por rol.
clients	{ organizationId: 1, name: 1 }	Búsqueda de clientes dentro de una organización.
equipment	{ organizationId: 1, serialNumber: 1 }	Localización de equipos por número de serie.
repair_orders	{ organizationId: 1, orderNumber: 1 } UNIQUE	Identificar una orden mediante número visible dentro del tenant.
repair_orders	{ organizationId: 1, status: 1 }	Listado de órdenes por estado.
repair_orders	{ organizationId: 1, assignedTechnicianId: 1, status: 1 }	Órdenes asignadas a un técnico.
repair_orders	{ organizationId: 1, clientId: 1 }	Historial de reparaciones de un cliente.
repair_orders	{ organizationId: 1, equipmentId: 1 }	Historial de reparaciones de un equipo.
repair_orders	{ organizationId: 1, createdAt: -1 }	Obtención de órdenes recientes.
audit_logs	{ organizationId: 1, createdAt: -1 }	Consulta cronológica de acciones de auditoría.

8.1. Validaciones preliminares

- Los campos obligatorios se validarán mediante DTOs del backend durante la etapa de implementación.
- El email del usuario deberá respetar un formato válido y será único globalmente en el MVP.
- El rol del usuario deberá pertenecer al conjunto de roles permitido.
- Una orden deberá pertenecer a una organización válida.
- Cliente, equipo y técnico asociados a una orden deberán pertenecer a la misma organización.
- Las transiciones de estado deberán respetar el flujo definido.
- OrderStatus y QuoteStatus deberán respetar las reglas de consistencia establecidas.
- El monto del presupuesto deberá ser mayor o igual a cero.
- Las operaciones deberán verificar autenticación y autorización antes de acceder a datos.
- La integridad de las referencias entre organización, cliente, equipo, técnico y orden será validada por la capa de servicio.

9. Módulos funcionales del MVP

El siguiente listado traduce el alcance aprobado en la primera entrega a módulos concretos para su desarrollo posterior. P0 identifica módulos críticos para el flujo principal del MVP; P1 identifica módulos importantes del alcance que pueden implementarse con menor prioridad relativa. La complejidad es preliminar y podrá ajustarse al refinar el backlog.

ID	Módulo	Descripción	Prioridad	Complejidad
MOD-01	Autenticación	Registro e inicio de sesión, generación y validación de JWT y acceso seguro a recursos protegidos.	P0	Alta
MOD-02	Organizaciones	Gestión de servicios técnicos y pertenencia de datos y usuarios, garantizando aislamiento lógico entre organizaciones.	P0	Alta
MOD-03	Usuarios y Roles	Gestión de usuarios y roles ADMIN, RECEPTIONIST y TECHNICIAN, con permisos según responsabilidades.	P1	Alta
MOD-04	Clientes	Registro, consulta y actualización de clientes de cada organización.	P0	Media
MOD-05	Equipos	Gestión de dispositivos y asociación con el cliente propietario.	P0	Media
MOD-06	Órdenes de Reparación	Gestión del trabajo de reparación, vinculando cliente, equipo, técnico responsable y ciclo de vida.	P0	Muy alta
MOD-07	Diagnóstico	Registro de falla detectada, solución propuesta y observaciones técnicas.	P0	Media
MOD-08	Presupuesto	Gestión de conceptos, importe y decisión del cliente: pendiente, aprobado o rechazado.	P0	Media/Alta
MOD-09	Estados e Historial	Control de estados y trazabilidad cronológica de cambios relevantes.	P0	Alta
MOD-10	Entrega	Cierre del ciclo de reparación y registro de entrega del equipo.	P0	Baja/Media
MOD-11	Dashboard	Indicadores operativos básicos sobre volumen de órdenes y distribución por estado.	P1	Media

10. Estructura del repositorio

La segunda entrega se mantiene en el mismo repositorio único declarado en la primera entrega. La documentación correspondiente al diseño del sistema se organiza dentro del directorio docs/, mientras que frontend/ y backend/ contendrán progresivamente la implementación de la aplicación.

La estructura se diferencia entre los archivos actualmente incorporados al repositorio y la organización prevista para la etapa de implementación.

10.1 Estructura actual del repositorio

La estructura actual contiene la documentación correspondiente a las entregas realizadas, el diseño de base de datos y la definición de módulos funcionales del MVP.

```
TFI-SaaS-Servicios-Tecnicos/
├── frontend/
├── backend/
├── docs/
│   ├── arquitectura/
│   │   └── modulos.md
│   ├── base-datos/
│   │   ├── modelo-documental.md
│   │   ├── colecciones.md
│   │   ├── indices.md
│   │   └── ejemplos.json
│   └── entregas/
│       ├── 01-primera-entrega/
│       │   └── TFI_Primer_Entrega.pdf
│       └── 02-segunda-entrega/
│           └── TFI_Segunda_Entrega_FixTrack.pdf
└── README.md
└── .gitignore
```

Criterio de organización

El **README** ubicado en la raíz funciona como presentación general y punto de acceso a la documentación del proyecto.

La carpeta **docs/arquitectura** contiene las definiciones arquitectónicas y funcionales del sistema. En esta etapa incorpora el archivo modulos.md con el listado de módulos del MVP.

La carpeta **docs/base-datos** contiene la documentación técnica del modelo MongoDB, incluyendo el modelo documental, colecciones, índices preliminares y ejemplos de documentos.

La carpeta **docs/entregas** conserva una copia de los documentos presentados en cada instancia académica.

10.2 Organización prevista para la implementación

Durante la etapa de desarrollo, frontend y backend se organizarán de manera modular. Esta estructura es preliminar y podrá ajustarse según las necesidades detectadas durante la implementación.

Frontend previsto:

```
frontend/  
└─ src/  
    └─ modules/  
        ├── auth/  
        ├── organizations/  
        ├── users/  
        ├── clients/  
        ├── equipment/  
        ├── repair-orders/  
        │   ├── diagnosis/  
        │   ├── quotes/  
        │   ├── status-history/  
        │   └── delivery/  
        └── dashboard/
```

Los módulos Diagnosis, Quote, StatusHistory y Delivery se representan dentro de repair-orders porque forman parte directamente del ciclo de vida de una orden de reparación.

Esto no significa que dejen de considerarse módulos funcionales del MVP, sino que su implementación estará estrechamente vinculada al agregado RepairOrder definido en el modelo documental.

Backend previsto:

```
backend/  
└─ src/  
    └─ main/  
        └─ java/  
            └─ .../  
                ├── auth/  
                ├── organization/  
                ├── user/  
                ├── client/  
                ├── equipment/  
                ├── repairorder/  
                └── dashboard/
```

La estructura del backend seguirá una arquitectura por capas. Cada dominio podrá organizar internamente responsabilidades de presentación, lógica de negocio, persistencia, modelos y transferencia de datos.

10.3. Organización prevista del módulo RepairOrder

RepairOrder constituye el agregado principal del sistema y concentra gran parte del ciclo operativo de una reparación.

La organización prevista para este módulo es:

```

repairorder/
├── controller/
│   └── RepairOrderController.java
├── service/
│   └── RepairOrderService.java
├── repository/
│   └── RepairOrderRepository.java
├── model/
│   ├── RepairOrder.java
│   ├── Diagnosis.java
│   ├── Quote.java
│   ├── StatusHistory.java
│   └── Delivery.java
├── dto/
│   ├── CreateRepairOrderRequest.java
│   ├── UpdateRepairOrderRequest.java
│   └── RepairOrderResponse.java
└── mapper/
    └── RepairOrderMapper.java
  
```

Esta organización refleja la arquitectura por capas prevista para el backend y mantiene dentro del agregado RepairOrder aquellos elementos que poseen una fuerte dependencia de su ciclo de vida.

La estructura representa una propuesta inicial de implementación y podrá refinarse durante el desarrollo sin modificar las responsabilidades funcionales definidas para el MVP.

11. Estrategia de validación y pruebas para la etapa de implementación

La presente entrega se concentra en análisis y diseño. La estrategia de pruebas se mantiene para que las decisiones actuales contemplen desde el inicio los casos críticos que deberán verificarse durante la implementación.

Nivel futuro	Herramientas previstas	Casos prioritarios
Backend unitario	JUnit 5 + Mockito	Reglas del ciclo de órdenes, presupuestos, transiciones de estado, validaciones y filtros por organizationId.
Seguridad/API	Spring Boot Test + MockMvc	Endpoints protegidos, respuestas 401/403, permisos por rol y acceso cruzado entre tenants.
Frontend	Vitest + React Testing Library	Formularios y renderizado de estados o permisos relevantes.

12. Decisiones de diseño

Decisión	Fundamento
MongoDB documental	La orden de reparación se modela como agregado con información jerárquica embebida.
repair_orders como agregado principal	Concentra el ciclo operativo de una reparación.
Diagnosis / Quote / StatusHistory / Delivery embebidos	Presentan dependencia fuerte del ciclo de vida de la orden.
Organization / User / Client / Equipment referenciados	Son entidades administradas independientemente.
organizationId transversal	Es la base del aislamiento lógico multi-organización.
Email globalmente único	Simplifica autenticación del MVP con email + contraseña.
orderNumber por tenant	Proporciona un identificador visible y práctico para las operaciones del negocio.
OrderStatus separado de QuoteStatus	Evita ambigüedades entre el estado operativo de la reparación y la decisión sobre el presupuesto.
Historial cronológico	Mantiene trazabilidad de cambios relevantes.
Índices orientados a consultas	Se priorizan búsquedas por organización, estado, técnico, cliente, equipo y fecha.
Arquitectura por capas	Separa responsabilidades del backend y facilita mantenimiento, seguridad y pruebas.

13. Límites de esta entrega

La presente entrega define análisis, diseño y estructura documental para el MVP. No implica que todas las funcionalidades se encuentren implementadas en esta instancia. La implementación continuará de acuerdo con el plan de trabajo y con las decisiones validadas por la tutora y el comité.

- Pagos online y suscripciones comerciales del SaaS.
- Aplicación móvil nativa.
- Integración oficial con WhatsApp, SMS u otros proveedores externos.
- Portal de autoservicio para clientes finales.
- Facturación electrónica o integración con controladores fiscales.
- Gestión avanzada de inventario de repuestos y compras.
- Reportes analíticos avanzados o funcionalidades de inteligencia artificial.

14. Checklist de cumplimiento de la Segunda Entrega

Ítem esperado	Dónde se encuentra	Estado
Esquema de base de datos	Secciones 3, 4 y 5	Completo
Colecciones, campos y tipos	Sección 4	Completo
Referencias y relaciones	Secciones 3.3 y 5	Completo
Índices principales	Sección 8	Completo
Validaciones preliminares	Sección 8.1	Completo
Listado de módulos	Sección 9	Completo
Arquitectura del proyecto	Sección 2	Completo
Tecnologías y justificación	Secciones 2.2 y 2.3	Completo
Aislamiento multi-organización	Sección 7	Completo
Estructura del repositorio	Sección 10	Completo
Delimitación del MVP	Sección 13	Completo

Conclusión

El diseño presentado establece una base documental coherente con el flujo aprobado para FixTrack, plataforma SaaS de gestión de servicios técnicos. MongoDB Atlas permitirá representar las órdenes como agregados que concentran diagnóstico, presupuesto, historial y entrega, mientras que organizaciones, usuarios, clientes y equipos se mantienen como documentos independientes relacionados mediante identificadores.

El modelo incorpora organizationId como regla transversal de aislamiento lógico, define de manera diferenciada y consistente el estado operativo de la orden y el estado del presupuesto, incorpora un número de orden visible por tenant y establece índices preliminares orientados a las consultas principales del MVP. La arquitectura cliente-servidor y por capas mantiene el stack definido en la primera entrega y deja preparado el proyecto para avanzar con la implementación incremental.